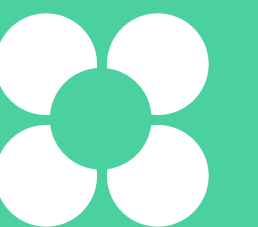


REST

Василий Дорохин
QA-engineer Quadcode



План занятия

- 1 REST
- 2 Валидация запросов
- 3 Работа с ошибками в Spring
- 4 WebMvcConfigurer

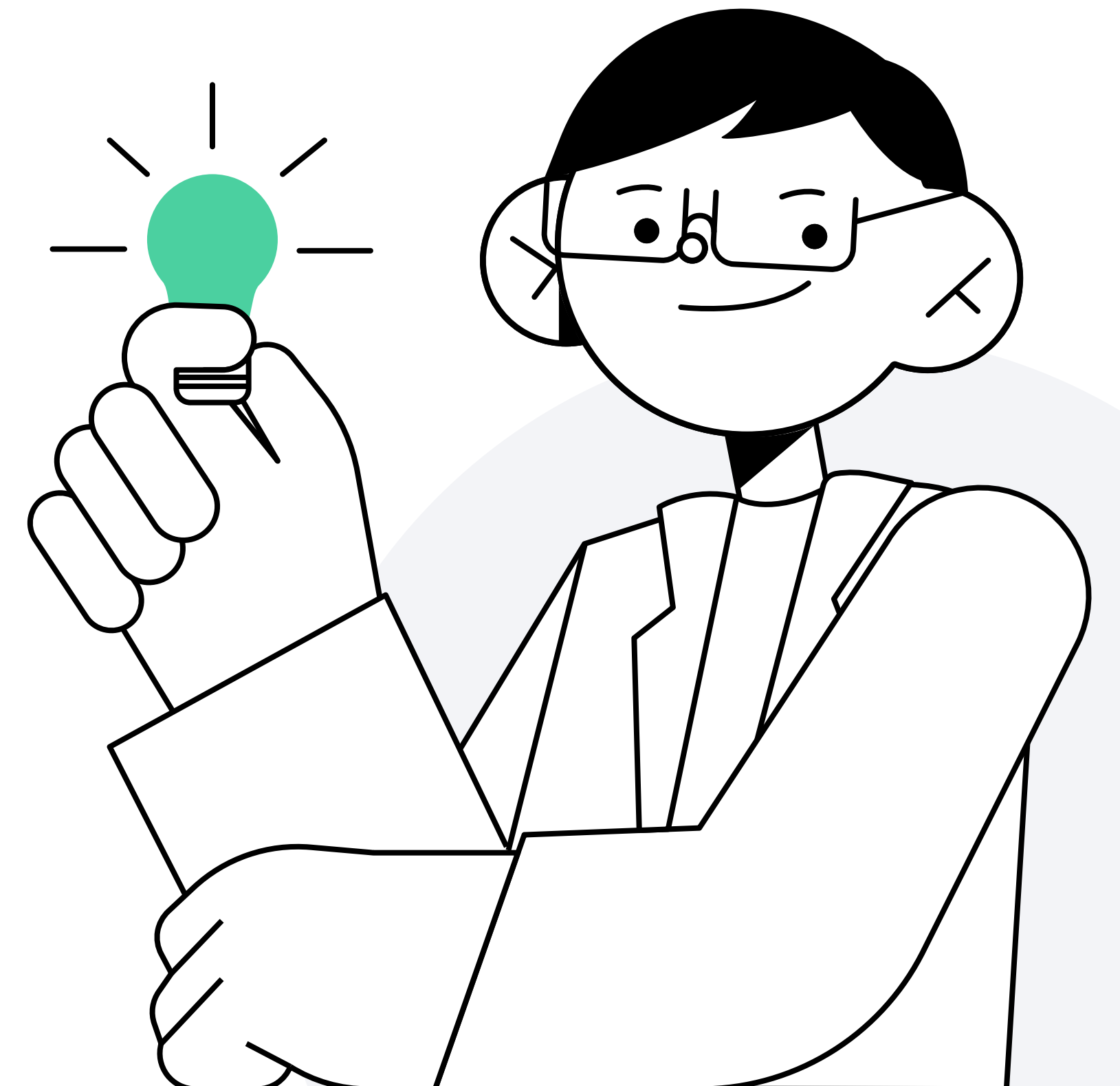


REST

Василий Дорохин
QA-engineer Quadcode

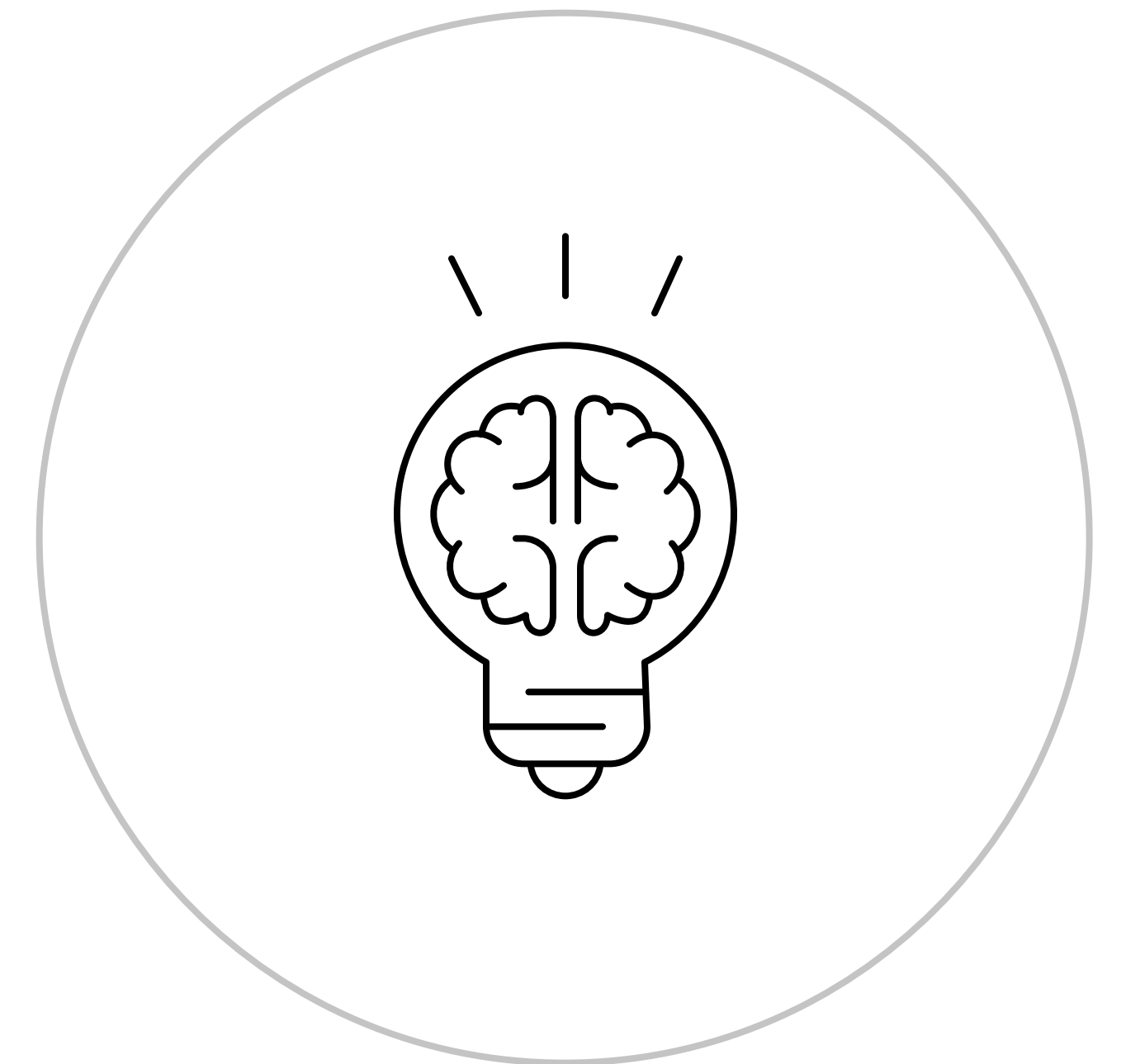
Тема занятия

- 1 Главные принципы REST
- 2 Основные аннотации для работы с REST в Spring



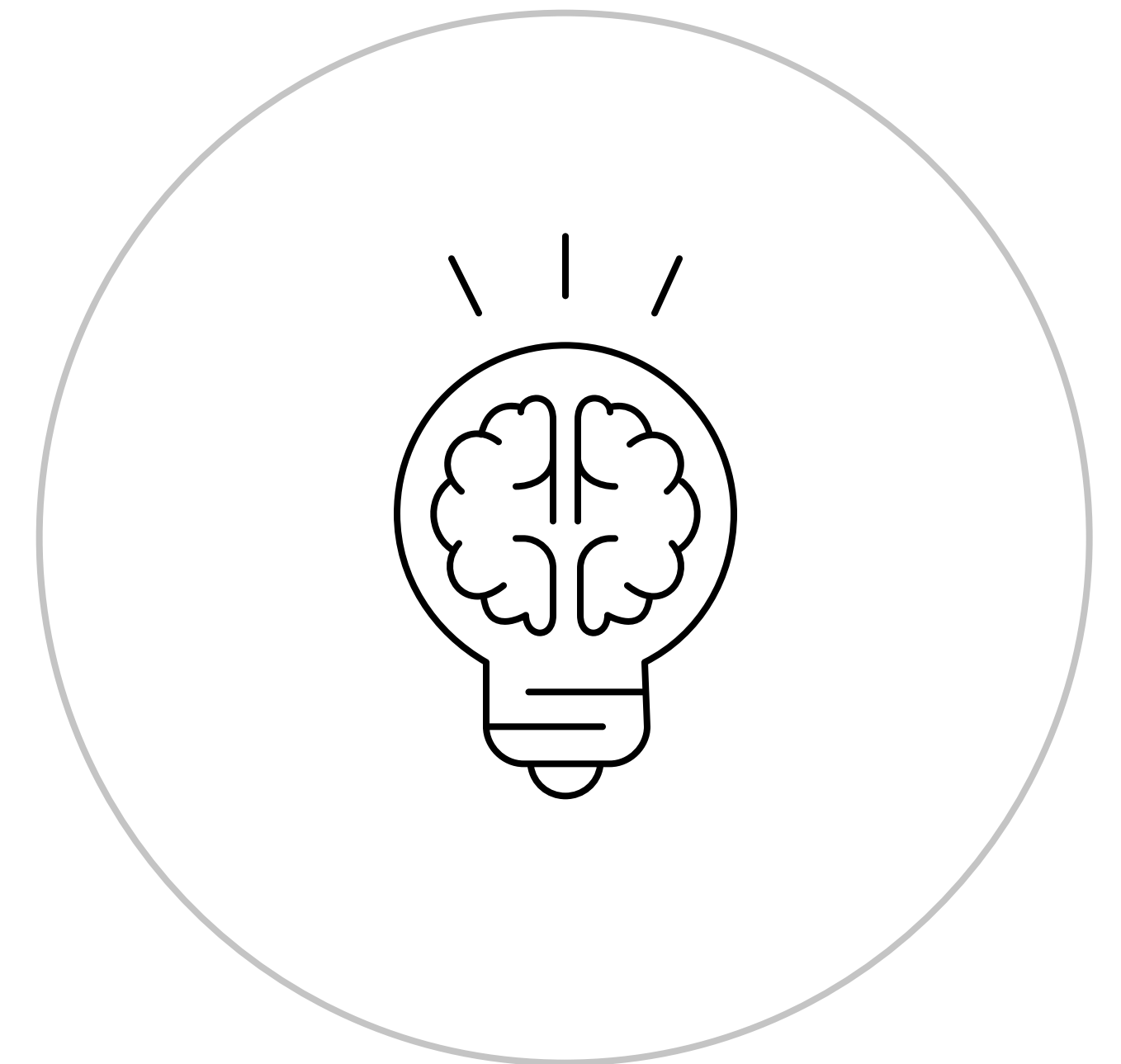
REST

REST (Representational State Transfer) – это архитектурный стиль взаимодействия между разными компонентами системы, которые могут находиться в разных местах



REST

RESTful-приложение — это приложение, которое построено с учётом наложенных архитектурой REST ограничений



REST

Главные принципы REST:

- архитектура клиент-сервер
- единообразный интерфейс
- отсутствие состояния
- слои
- кэширование
- код по запросу



REST

Основные аннотации для работы с REST в Spring:

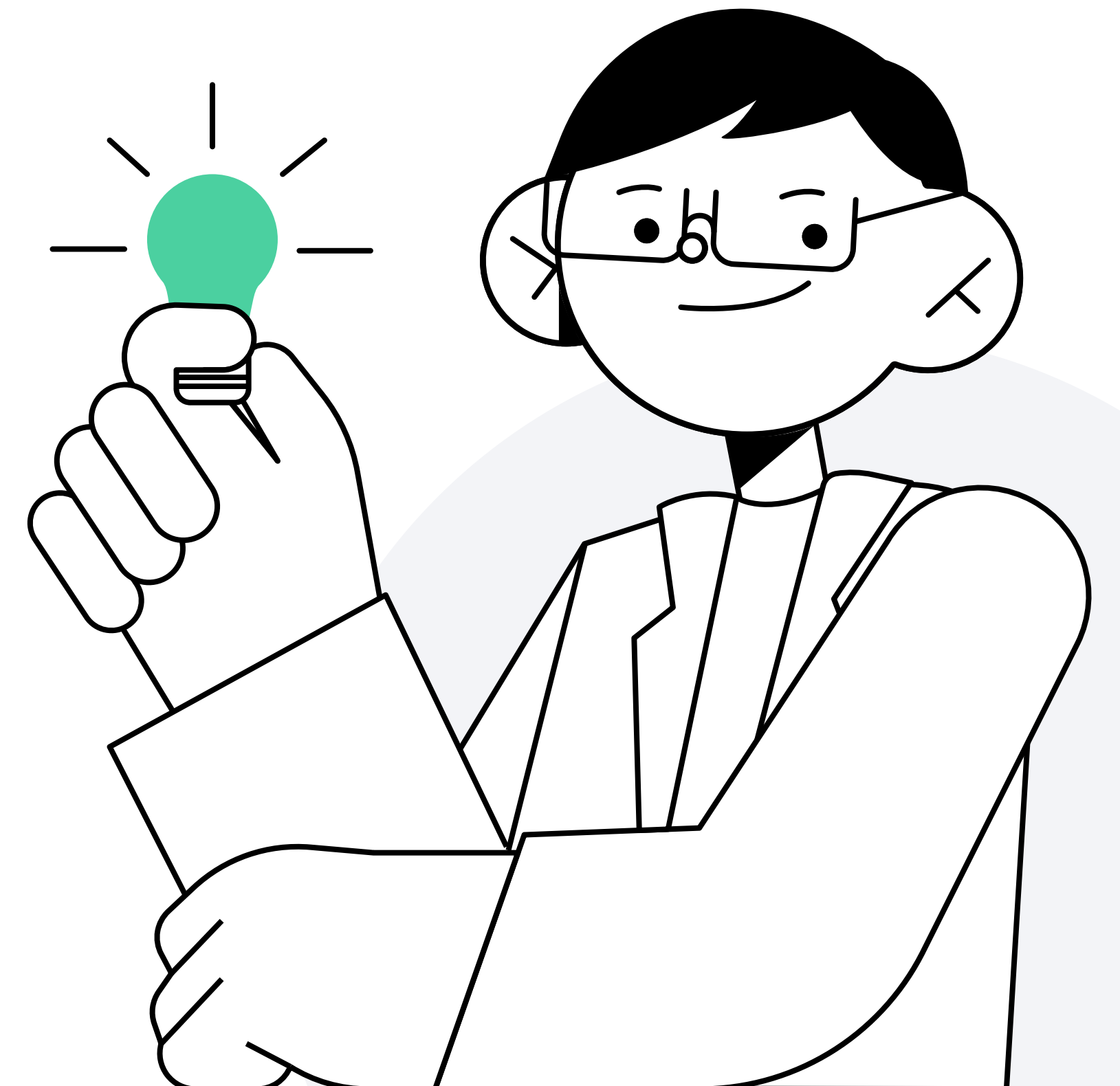
- @RestController. Указывает на класс-обработчик пользовательских запросов
- @RequestMapping. Формирует соответствие путь<->обработчик
- @RequestBody. Указывает, что надо десериализовать объект из тела запроса
- @RequestParam. Указывает, что надо десериализовать объект из query запроса
- @PathVariable. Указывает, что надо десериализовать объект из пути запроса

Работа с ошибками в Spring

Василий Дорохин
QA-engineer Quadcode

Тема занятия

- 1 Механизмы обработки ошибок
- 2 Интерфейс HandlerExceptionResolver



HandlerExceptionResolver

В Spring реализации интерфейса **HandlerExceptionResolver** обрабатывают неожиданные исключения, возникающие во время выполнения методов контроллера:

```
public interface HandlerExceptionResolver {  
  
    /** Try to resolve the given exception that got thrown during handler execution, ...*/  
    @Nullable  
    ModelAndView resolveException(  
        HttpServletRequest request, HttpServletResponse response, @Nullable Object handler, Exception ex);  
}
```

HandlerExceptionResolver

Spring предоставляет несколько базовых реализаций интерфейса `HandlerExceptionResolver`:

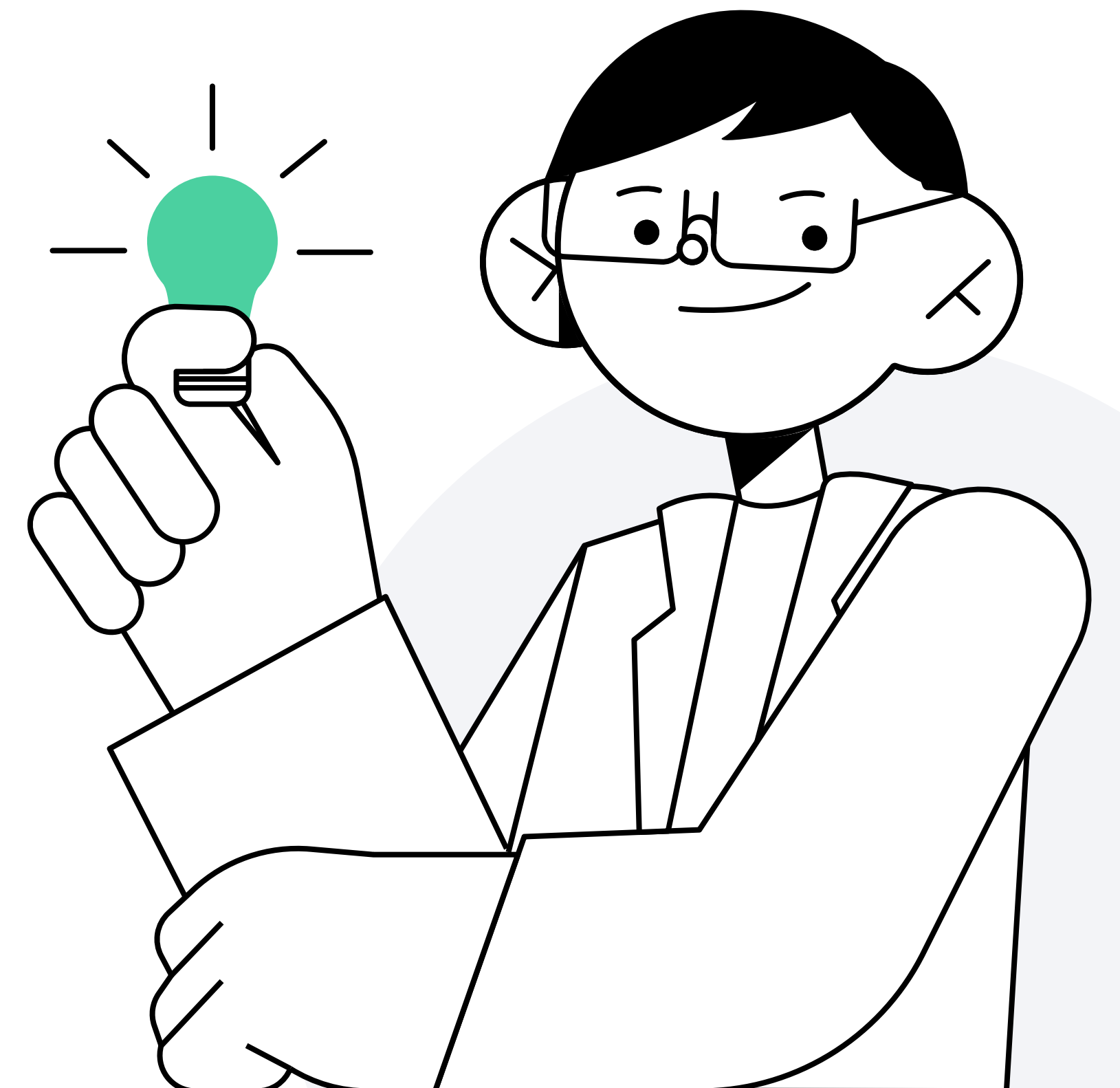
- `SimpleMappingExceptionResolver`. Позволяют декларативно мапить исключения в определённые представления вместе с некоторой дополнительной логикой
- `DefaultHandlerExceptionResolver`. Переводит стандартные исключения Spring в REST-статусы для ответа
- `ExceptionHandlerExceptionResolver`. Позволяет обрабатывать ошибки в контроллерах на основе типа ошибки

Валидация запросов

Василий Дорохин
QA-engineer Quadcode

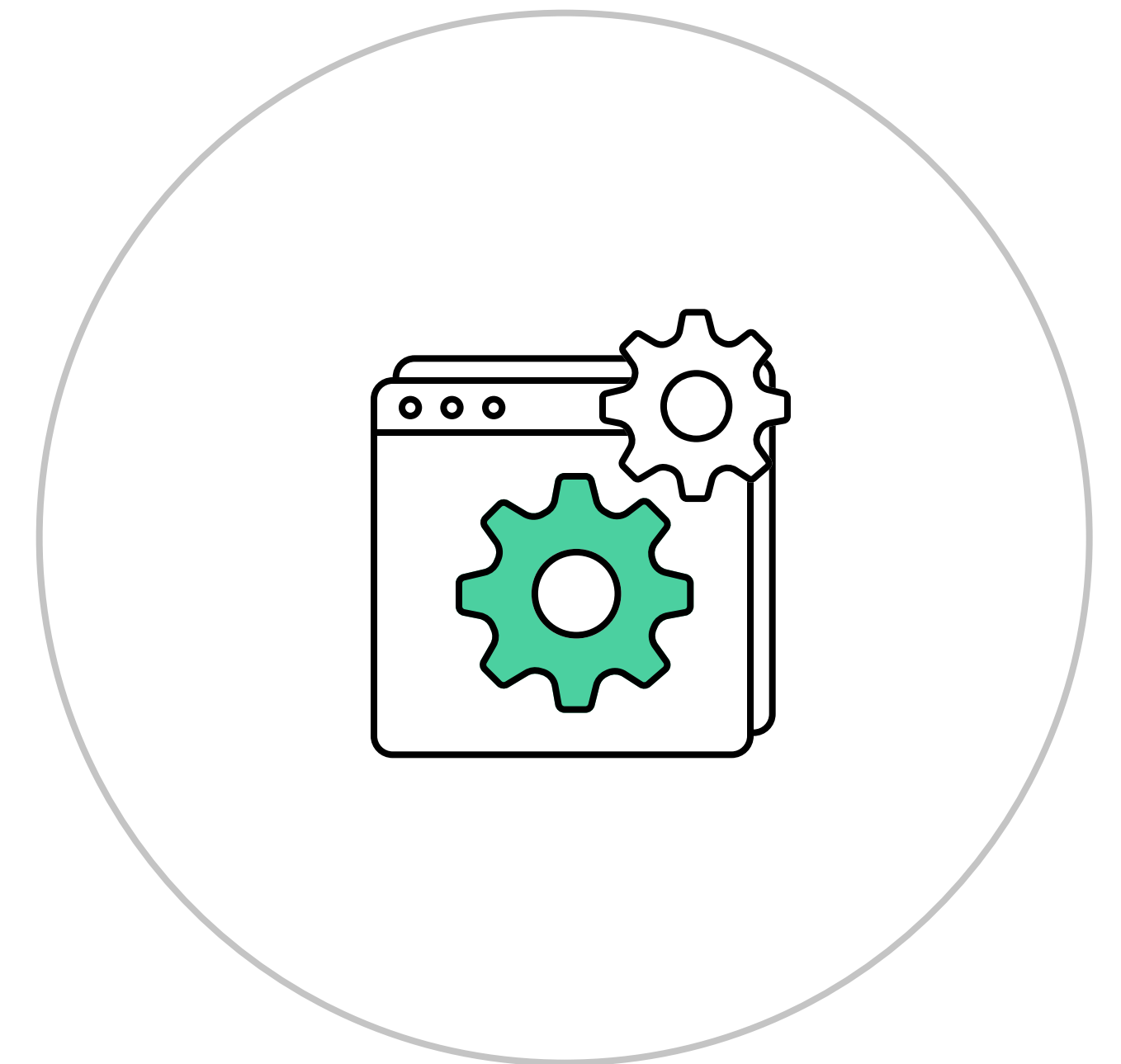
Тема занятия

→ Валидация полей объектов и объектов методов



Валидация запросов

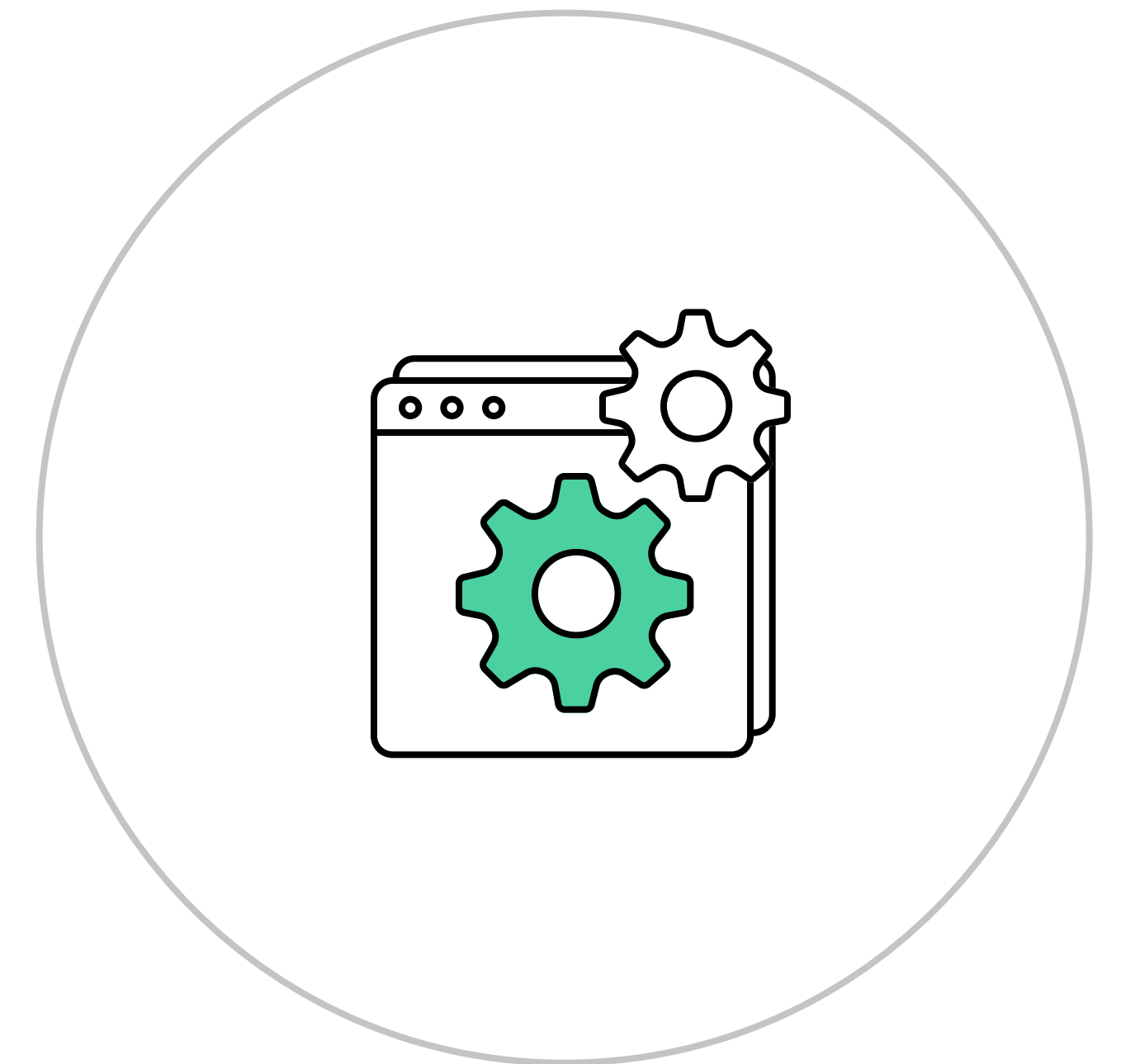
Для валидации объектов запросов в контроллерах Spring использует predetermined бины Java Validation API.



Валидация запросов

Для валидации объектов запросов в контроллерах Spring использует predetermined бины Java Validation API.

Для этого либо контроллер, либо объект, который валидируем, должны быть помечены аннотацией **@Validated**

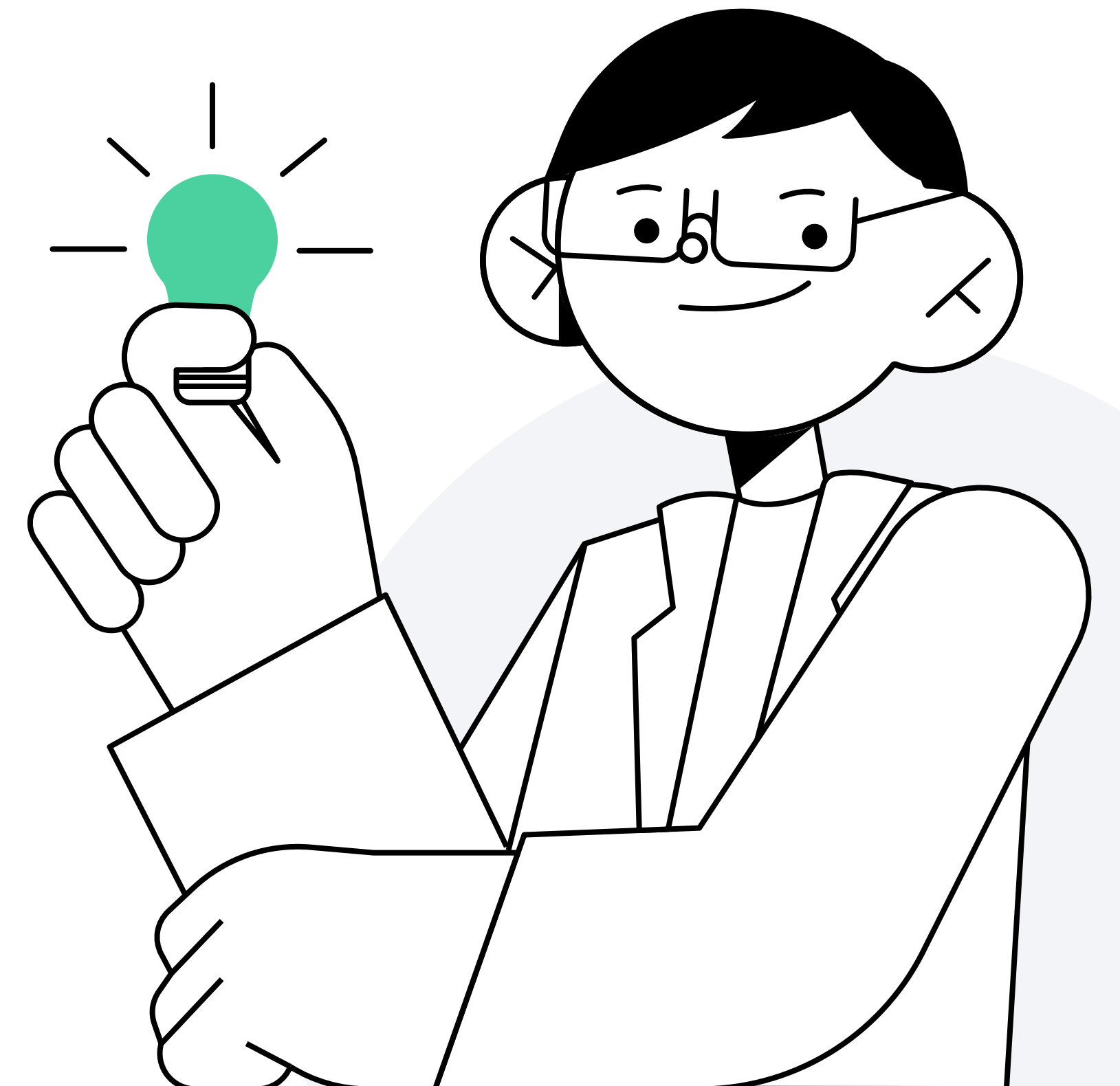


WebMvcConfigurer

Василий Дорохин
QA-engineer Quadcode

Тема занятия

- 1 Rest в Spring boot
- 2 WebMvcConfigurer



REST в Spring boot

При помощи автоконфигураций создается множество сконфигурированных бинов:

→ RequestMappingHandlerAdapter

→ DispatcherServlet

→ прочие бины



WebMvcConfigurer

WebMvcConfigurer — это интерфейс, с помощью которого можно конфигурировать то, как будет ваше приложение работать с REST

WebMvcConfigurer

WebMvcConfigurer может:

- регистрировать Interceptor классы, которые могут добавлять дополнительную логику перед и после выполнения методов контроллера
- регистрировать контроллер для возвращения статичных ресурсов в ответ на запрос
- **создавать кастомные конвертеры объектов** для пользовательских типов данных, которые не могут быть десериализованы стандартным образом

Пользовательский конвертер

Для работы пользовательских конвертеров необходимо реализовать интерфейс `Converter` и зарегистрировать его:

```
@Configuration
public class RulesWebMvcConfig implements WebMvcConfigurer {
    private static final Map<String, InstrumentType> types = Stream.of(Bond.values(), Equity.values())
        .flatMap(Stream::of).collect(Collectors.toMap(InstrumentType::name, Function.identity()));

    @Override
    public void addFormatters(FormatterRegistry registry) {
        registry.addConverter(new SpringRateFeatureConverter());
    }

    private static class SpringRateFeatureConverter implements Converter<String, InstrumentType> {
        @Override
        public InstrumentType convert(String source) {
            return types.getOrDefault(source, Unknown.UNKNOWN);
        }
    }
}
```

Регистрация статических данных

Для удобной работы со статичным ресурсами необходимо поставить в соответствие относительный путь, по которому надо вернуть ресурс, и название и путь ресурса, который Spring будет искать в папке `resources/`<путь к ресурсу и его название>:

```
@Configuration
public class StaticResourceMapping implements WebMvcConfigurer {
    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        String index = "index";
        Consumer<String> registerViewController = path -> registry.addViewController( urlPath: "/" + path).setViewName(index);
        Stream.of("/", "ui", index).forEach(registerViewController);
    }
}
```

Пользовательские аннотации

```
public class FeatureGroupWebMvcConfig implements WebMvcConfigurer {
    private static final Map<String, InstrumentType> types = Stream.of(Bond.values(), Equity.values())
        .flatMap(Stream::of).collect(Collectors.toMap(InstrumentType::name, Function.identity()));

    @Override
    public void addArgumentResolvers(List<HandlerMethodArgumentResolver> resolvers) {
        resolvers.add(new CustomParamResolver());
    }

    private class CustomParamResolver implements HandlerMethodArgumentResolver {
        @Override
        public boolean supportsParameter(MethodParameter parameter) {
            return parameter.hasParameterAnnotation(Instrument.class);
        }

        @Override
        public Object resolveArgument(MethodParameter parameter, ModelAndViewContainer mavContainer,
            NativeWebRequest webRequest, WebDataBinderFactory binderFactory) {
            String feature = webRequest.getParameter(paramName: "feature");

            return types.getOrDefault(feature, Unknown.UNKNOWN);
        }
    }
}
```

Итоги

- Spring дает нам много возможностей для работы с REST окружением нашего приложения. Он дает нам удобно управлять потоком выполнения программы с помощью перехватчиков exception'ов как локально, так и глобально. Также, он позволяет нам удобно валидировать и конфигурировать наши пользовательские типы данных